Solution Architect — Detailed Interview Questionnaire & Answers

Part 2: Security, Storage, Databases & Infrastructure as Code

---

Section 4: AWS Security

Q14. Explain IAM in depth — Users, Roles, Policies, and best practices.

Answer:

IAM Components:

Component	Purpose	Example
User	Human identity with long-lived credentials	Developer account (avoid in production)
Group	Collection of users sharing permissions	"Developers" group, "Admins" group
Role	Assumed identity for services/temporary access	EC2 instance role, Lambda execution role, cross-account role
Policy	JSON document defining permissions	S3ReadOnly, AdministratorAccess

Policy Types:

- **AWS Managed:** Pre-built by AWS (e.g., `AmazonS3ReadOnlyAccess`)
- **Customer Managed:** Custom policies you create and version
- **Inline:** Embedded directly in a user/group/role (avoid — hard to manage)

Policy Evaluation Logic:

```
Explicit Deny > Explicit Allow > Implicit Deny (default)
```

Best Practices I Follow:

1. **No IAM Users for applications** — Use IAM Roles everywhere
2. **Least privilege:** Start with zero permissions, add only what's needed
3. **Use IAM Access Analyzer** to identify unused permissions and tighten policies
4. **SCP (Service Control Policies)** at the Organization level to set guardrails
5. **Conditions in policies:** Restrict by IP, VPC endpoint, MFA, time of day
6. **Cross-account access:** Use `sts:AssumeRole` with external ID, not shared credentials

7. **No wildcards (*)** on actions or resources in production policies
8. **Rotate credentials** — Use AWS STS for temporary credentials (1-12 hour sessions)

Terraform Example — Least-Privilege Lambda Role:

```
resource "aws_iam_role" "lambda_exec" {
  name = "bedrock-query-lambda-role"
  assume_role_policy = jsonencode({
    Version = "2012-10-17"
    Statement = [{
      Action    = "sts:AssumeRole"
      Effect    = "Allow"
      Principal = { Service = "lambda.amazonaws.com" }
    }]
  })
}

resource "aws_iam_role_policy" "lambda_bedrock" {
  role = aws_iam_role.lambda_exec.id
  policy = jsonencode({
    Version = "2012-10-17"
    Statement = [
      {
        Effect   = "Allow"
        Action   = ["bedrock:InvokeModel"]
        Resource = "arn:aws:bedrock:us-east-1::foundation-
model/anthropic.claude-*"
      },
      {
        Effect   = "Allow"
        Action   = ["s3:GetObject"]
        Resource = "${aws_s3_bucket.knowledge.arn}/*"
      },
      {
        Effect   = "Allow"
        Action   = ["logs:CreateLogGroup", "logs:CreateLogStream",
"logs:PutLogEvents"]
        Resource = "arn:aws:logs:*:*:*"
      }
    ]
  })
}
```

Q15. How do you implement encryption in AWS? Explain at-rest and in-transit encryption.

Answer:

Encryption At-Rest:

Service	Encryption Method	Key Management
S3	SSE-S3, SSE-KMS, SSE-C	AWS managed, CMK, customer-provided
RDS	AES-256 via KMS	Must enable at creation (cannot add later)
EBS	AES-256 via KMS	Enable by default in account settings
DynamoDB	AWS owned keys or CMK	Default encrypted, upgrade to CMK for control
EKS Secrets	Envelope encryption via KMS	Enable secrets encryption in cluster config
Lambda env vars	KMS encryption	Automatic with option for CMK

Encryption In-Transit:

Layer	Method
Client → CloudFront	TLS 1.2/1.3 with ACM certificate
CloudFront → ALB	TLS with custom SSL certificate
ALB → Application	TLS or plain HTTP (within VPC)
App → RDS	`require_ssl` parameter + rds-ca certificate
App → ElastiCache	TLS enabled on Redis cluster
Service-to-Service (EKS)	mTLS via Istio service mesh

KMS Key Hierarchy:

```
AWS KMS
├── AWS Managed Keys (aws/s3, aws/rds) — Free, AWS rotates
├── Customer Managed Keys (CMK) — $1/month + API calls
│   ├── Symmetric (AES-256) — Most common, for envelope encryption
│   └── Asymmetric (RSA/ECC) — For sign/verify, external encryption
└── Key Policies — Control who can use/manage the key
```

My Approach (Bedrock RAG project):

```
resource "aws_kms_key" "main" {
  description             = "CMK for Bedrock RAG encryption"
  deletion_window_in_days = 30
  enable_key_rotation     = true  # Auto-rotate annually

  policy = jsonencode({
    Statement = [
      {
        Sid       = "AllowKeyAdministration"
        Effect    = "Allow"
        Principal = { AWS =
"arn:aws:iam::${data.aws_caller_identity.current.account_id}:root" }
        Action    = "kms:*"
        Resource  = "*"
      },
      {
        Sid       = "AllowLambdaToUseKey"
        Effect    = "Allow"
        Principal = { AWS = aws_iam_role.lambda_exec.arn }
        Action    = ["kms:Decrypt", "kms:GenerateDataKey"]
        Resource  = "*"
      }
    ]
  })
}
```

Q16. What AWS security services do you use for threat detection and compliance?

Answer:

Service	Purpose	What It Detects
GuardDuty	Threat detection	Compromised instances, unusual API calls, crypto-mining, DNS exfiltration

Service	Purpose	What It Detects
Security Hub	Centralized security posture	Aggregates findings from GuardDuty, Inspector, Config, Macie
CloudTrail	API audit log	Who did what, when, from where (every API call logged)
AWS Config	Configuration compliance	Drift detection, compliance rules (e.g., "all S3 buckets must be encrypted")
Inspector	Vulnerability scanning	CVEs in EC2/ECR images, network exposure
Macie	Data classification	PII in S3 buckets, sensitive data discovery
IAM Access Analyzer	Access analysis	External access, unused permissions, policy validation
WAF	Web application firewall	SQL injection, XSS, bot protection, rate limiting

My Standard Security Stack (Terraform):

```
GuardDuty → SNS → Lambda → Slack notification
CloudTrail → S3 (encrypted) + CloudWatch Logs
Config Rules → "s3-bucket-encryption", "rds-encryption-enabled", "iam-no-inline-policy"
Security Hub → Dashboard for CIS Benchmark compliance
```

Bedrock Guardrails (for AI security):

- Content filtering (harmful content blocked)
- PII redaction (SSN, credit card, phone numbers automatically masked)
- Topic denial (prevent model from discussing out-of-scope topics)
- Word/phrase filters (block specific terms)

Q17. How do you manage secrets in AWS?

Answer:

Option 1: AWS Secrets Manager (recommended for sensitive credentials)

- Stores database passwords, API keys, OAuth tokens
- **Auto-rotation:** Built-in Lambda-based rotation for RDS, Redshift, DocumentDB
- **Cross-account sharing** via resource policies
- **Cost:** $0.40/secret/month + 0.05/10K API calls

Option 2: AWS Systems Manager Parameter Store

- Stores configuration values and secrets
- **Standard parameters:** Free, up to 10K parameters
- **Advanced parameters:** $0.05/parameter/month, larger size, expiration policies
- **SecureString** type uses KMS encryption

When to use which:

Use Case	Secrets Manager	Parameter Store
DB passwords with auto-rotation	✅	❌
API keys, tokens	✅	✅ (SecureString)
Non-sensitive config (feature flags)	❌ (overkill)	✅ (String type)
Cost-sensitive	❌ ($0.40/secret)	✅ (free tier)
Cross-account sharing	✅	❌

In Terraform:

```
# Secrets Manager for DB password
resource "aws_secretsmanager_secret" "db_password" {
  name = "prod/rds/master-password"
  kms_key_id = aws_kms_key.main.id
}

# Parameter Store for app config
resource "aws_ssm_parameter" "api_url" {
  name  = "/prod/app/api-url"
  type  = "String"
  value = "https://api.example.com"
}

# Never hardcode secrets — reference dynamically
data "aws_secretsmanager_secret_version" "db" {
  secret_id = aws_secretsmanager_secret.db_password.id
}
```

Section 5: AWS Storage

Q18. Compare S3 storage classes and explain lifecycle policies.

Answer:

Storage Class	Use Case	Availability	Min Duration	Cost (per GB/month)
S3 Standard	Frequently accessed data	99.99%	None	$0.023
S3 Intelligent-Tiering	Unknown access pattern	99.9%	None	$0.023 + monitoring fee
S3 Standard-IA	Infrequently accessed, rapid retrieval	99.9%	30 days	$0.0125
S3 One Zone-IA	Non-critical, infrequent	99.5% (single AZ)	30 days	$0.01
S3 Glacier Instant	Archive with millisecond retrieval	99.9%	90 days	$0.004
S3 Glacier Flexible	Archive, 1-12 hour retrieval	99.99%	90 days	$0.0036
S3 Glacier Deep Archive	Long-term archive, 12-48 hour retrieval	99.99%	180 days	$0.00099

Lifecycle Policy Example:

```
{
  "Rules": [
    {
      "ID": "ArchiveOldLogs",
      "Status": "Enabled",
      "Filter": { "Prefix": "logs/" },
      "Transitions": [
```

```
        { "Days": 30,  "StorageClass": "STANDARD_IA" },
        { "Days": 90,  "StorageClass": "GLACIER" },
        { "Days": 365, "StorageClass": "DEEP_ARCHIVE" }
      ],
      "Expiration": { "Days": 2555 }  // Delete after 7 years
    }
  ]
}
```

S3 Security Best Practices:

- Block all public access (default since 2023)
- Enable versioning for critical buckets
- Enable server-side encryption (SSE-KMS with CMK)
- Enable access logging → separate logging bucket
- Use bucket policies + IAM policies (defense in depth)
- Use VPC endpoint for private access
- Enable MFA Delete for production buckets

Q19. When would you use EFS vs EBS vs S3?

Answer:

Feature	S3	EBS	EFS
Type	Object storage	Block storage	File storage (NFS)
Access	HTTP/API	Single EC2 (or multi-attach io2)	Multiple EC2/ECS/EKS/Lambda
Performance	High throughput	High IOPS (io2: 64K IOPS)	Throughput-optimized or IOPS-optimized
Durability	99.999999999% (11 9's)	99.999%	99.999999999%
Use Case	Static assets, backups, data lake	Database volumes, boot volumes	Shared config, CMS content, ML training data
Pricing	$0.023/GB	$0.08/GB (gp3)	$0.30/GB (Standard)

Feature	S3	EBS	EFS
Max Size	Unlimited	64 TB per volume	Unlimited (auto-grows)

Decision Framework:

```
Need shared access across multiple instances/containers?
  ├── YES → EFS (NFS mount)
  └── NO → Need low-latency block storage (database, OS)?
        ├── YES → EBS (gp3 for general, io2 for high IOPS)
        └── NO → S3 (object storage, most cost-effective)
```

EKS-Specific Storage:

- **EBS CSI Driver:** For StatefulSets (databases) — `gp3` StorageClass
- **EFS CSI Driver:** For shared volumes across pods (e.g., ML model files)
- **S3 CSI Driver (Mountpoint):** Read-heavy access to S3 data lakes

---

Section 6: AWS Databases

Q20. Compare RDS, Aurora, and DynamoDB. How do you choose between them?

Answer:

Feature	RDS	Aurora	DynamoDB
Type	Managed relational	Cloud-native relational	Managed NoSQL (key-value + document)
Engines	MySQL, PostgreSQL, MariaDB, Oracle, SQL Server	MySQL-compatible, PostgreSQL-compatible	Proprietary

Feature	RDS	Aurora	DynamoDB
Scaling	Vertical (instance size) + Read Replicas	Auto-scaling storage + up to 15 read replicas	Auto-scaling (on-demand or provisioned)
Storage	Up to 64 TB	Auto-grows up to 128 TB	Unlimited
Availability	Multi-AZ (standby)	Multi-AZ (6 copies across 3 AZs)	Multi-AZ by default, Global Tables for multi-region
Failover	60-120 seconds	< 30 seconds	No failover needed (serverless)
Cost	$	$$ (20% more than RDS MySQL)	Pay-per-request or provisioned
Serverless	❌	Aurora Serverless v2	On-Demand mode

Decision Framework:

```
Is data relational with complex queries/joins?
  ├── YES → Need high availability with fast failover?
  │     ├── YES → Aurora (6-way replication, <30s failover)
  │     └── NO → RDS (lower cost, simpler)
  └── NO → Key-value / document access patterns?
        ├── YES → DynamoDB (millisecond latency, infinite scale)
        └── NO → Need full-text search? → OpenSearch
                 Need graph queries? → Neptune
                 Need time-series? → Timestream
```

DynamoDB Design Principles:

- Design for **access patterns first**, not data normalization
- Use **single-table design** where possible
- **Partition Key** = high cardinality (e.g., user_id, order_id)
- **Sort Key** = for range queries (e.g., timestamp, status#timestamp)
- **GSI (Global Secondary Index)** for alternative access patterns
- Use **DynamoDB Streams** for event-driven architectures (trigger Lambda on data changes)

Q21. How do you implement caching in AWS? Explain caching strategies.

Answer:

Caching Layers:

```
Client → CloudFront (edge cache)
    → API Gateway (response cache)
        → Application (in-memory / ElastiCache)
            → Database
```

ElastiCache Options:

Feature	Redis	Memcached
Data Structures	Strings, Lists, Sets, Hashes, Sorted Sets, Streams	Simple key-value
Persistence	Snapshots + AOF	None (pure cache)
Replication	Multi-AZ with auto-failover	No replication
Cluster Mode	Yes (sharding)	Yes (sharding)
Use Case	Sessions, leaderboards, queues, pub/sub	Simple caching, session store

Caching Strategies:

1. **Cache-Aside (Lazy Loading):**
    - App checks cache → miss → query DB → write to cache → return
    - Pros: Only caches what's needed
    - Cons: Cache miss penalty (3 trips)
2. **Write-Through:**
    - App writes to cache AND DB simultaneously
    - Pros: Cache always consistent
    - Cons: Write penalty, caches unused data
3. **Write-Behind (Write-Back):**

- App writes to cache → cache async writes to DB
- Pros: Fast writes
- Cons: Risk of data loss if cache fails

4. **TTL-Based Expiration:**

   - Set TTL on cache entries (e.g., 300 seconds)
   - Balance between freshness and performance

My recommendation: Use **Cache-Aside with TTL** for most web applications. Add **Write-Through** for critical data where consistency matters.

Section 7: Infrastructure as Code — Terraform

Q22. Why Terraform over CloudFormation? Explain Terraform's architecture.

Answer:

Terraform vs CloudFormation:

Aspect	Terraform	CloudFormation
Provider	Multi-cloud (AWS, Azure, GCP, K8s, etc.)	AWS only
Language	HCL (HashiCorp Configuration Language)	JSON / YAML
State	External state file (S3 + DynamoDB)	Managed by AWS
Drift Detection	`terraform plan`	CloudFormation drift detection
Modularity	Modules (reusable, versioned)	Nested stacks (complex)

Aspect	Terraform	CloudFormation
Import	`terraform import`	`aws cloudformation import`
Ecosystem	Terraform Registry, thousands of providers	AWS-only resources
Preview	`terraform plan` (detailed diff)	Change Sets

Why I prefer Terraform:

1. **Multi-provider:** Manage AWS + Kubernetes + Helm + GitHub in one codebase
2. **HCL is more readable** than JSON/YAML
3. **Module ecosystem:** Reuse community modules from registry
4. **Plan output** is clearer and more actionable
5. **State management** is explicit and controllable

Terraform Architecture:

```
Configuration (.tf files)
    ↓
terraform init (download providers, modules)
    ↓
terraform plan (compare desired state vs current state)
    ↓
terraform apply (make API calls to reach desired state)
    ↓
State File (terraform.tfstate) — stored in S3 with DynamoDB locking
```

Core Concepts:

- **Provider:** Plugin that interacts with an API (aws, kubernetes, helm)
- **Resource:** Infrastructure object (aws_instance, aws_s3_bucket)
- **Data Source:** Read existing infrastructure (data.aws_vpc, data.aws_ami)
- **Module:** Reusable collection of resources
- **State:** Mapping between config and real-world resources
- **Backend:** Where state is stored (local, S3, Terraform Cloud)

Q23. How do you structure a production Terraform project?

Answer:

My Standard Project Structure:

```
terraform/
├── backend.tf              # S3 backend + DynamoDB locking
├── provider.tf             # AWS provider configuration
├── variables.tf            # Root-level input variables
├── outputs.tf              # Root-level outputs
├── main.tf                 # Module composition
├── terraform.tfvars        # Environment-specific values (gitignored)
│
├── modules/
│   ├── networking/
│   │   ├── main.tf         # VPC, subnets, IGW, NAT, endpoints
│   │   ├── variables.tf
│   │   └── outputs.tf
│   │
│   ├── iam/
│   │   ├── main.tf         # Roles, policies
│   │   ├── variables.tf
│   │   └── outputs.tf
│   │
│   ├── compute/            # ECS/EKS/Lambda
│   │   ├── main.tf
│   │   ├── variables.tf
│   │   └── outputs.tf
│   │
│   ├── database/           # RDS/DynamoDB
│   │   ├── main.tf
│   │   ├── variables.tf
│   │   └── outputs.tf
│   │
│   └── monitoring/         # CloudWatch, SNS
│       ├── main.tf
│       ├── variables.tf
│       └── outputs.tf
│
└── environments/           # Environment-specific tfvars
    ├── dev.tfvars
    ├── staging.tfvars
    └── prod.tfvars
```

Key Principles:

1. **One module per logical component** (networking, compute, database)
2. **Modules communicate via outputs/inputs** (loose coupling)
3. **Environment separation** via workspaces or separate state files
4. **Remote state** in S3 with DynamoDB locking
5. **Version pin everything** — providers, modules, Terraform itself

6. **Use `terraform fmt` and `terraform validate`** in CI pipeline

Q24. Explain Terraform state management. What happens if state is corrupted?

Answer:

What is State?

- JSON file mapping your Terraform config to real-world resource IDs
- Without state, Terraform doesn't know what it already created
- Contains sensitive data (DB passwords, keys) — must be encrypted

Remote State Best Practice:

```
terraform {
  backend "s3" {
    bucket         = "myproject-terraform-state"
    key            = "prod/terraform.tfstate"
    region         = "us-east-1"
    encrypt        = true
    kms_key_id     = "alias/terraform-state-key"
    dynamodb_table = "terraform-state-lock"  # Prevents concurrent
modifications
  }
}
```

State Operations:

```
terraform state list              # List all resources in state
terraform state show aws_s3_bucket.main  # Show details of one resource
terraform state mv                # Rename/move resources without destroy
terraform state rm                # Remove resource from state (keeps real
resource)
terraform import                  # Import existing resource into state
terraform state pull              # Download remote state locally
terraform state push              # Upload local state to remote
```

If State is Corrupted:

1. **S3 versioning saves you** — recover previous version from S3 version history
2. **If no versioning:** Use `terraform import` to rebuild state resource by resource

3. **If partial corruption:** `terraform state rm` the corrupted resource, then `terraform import` it back
4. **Nuclear option:** Delete state, re-import everything (painful but possible)

Prevention:

- ✅ Enable S3 versioning on state bucket
- ✅ Enable DynamoDB locking (prevents concurrent `terraform apply`)
- ✅ Never edit state manually
- ✅ Use `terraform state mv` instead of deleting/re-creating
- ✅ CI/CD pipeline is the only way to run `terraform apply` in prod

Q25. Explain Terraform modules — how do you write and version them?

Answer:

Module Structure:

```
# modules/networking/main.tf
resource "aws_vpc" "main" {
  cidr_block           = var.vpc_cidr
  enable_dns_support   = true
  enable_dns_hostnames = true

  tags = merge(var.tags, { Name = "${var.project}-vpc" })
}

resource "aws_subnet" "private" {
  count             = length(var.private_subnet_cidrs)
  vpc_id            = aws_vpc.main.id
  cidr_block        = var.private_subnet_cidrs[count.index]
  availability_zone = var.availability_zones[count.index]

  tags = merge(var.tags, {
    Name = "${var.project}-private-${count.index + 1}"
    "kubernetes.io/role/internal-elb" = "1"
  })
}

# modules/networking/variables.tf
variable "vpc_cidr" {
  description = "CIDR block for VPC"
  type        = string
  default     = "10.0.0.0/16"
}
```

```
variable "private_subnet_cidrs" {
  description = "CIDR blocks for private subnets"
  type        = list(string)
}

# modules/networking/outputs.tf
output "vpc_id" {
  value = aws_vpc.main.id
}

output "private_subnet_ids" {
  value = aws_subnet.private[*].id
}
```

Using the Module:

```
# main.tf (root)
module "networking" {
  source = "./modules/networking"

  vpc_cidr             = "10.0.0.0/16"
  private_subnet_cidrs = ["10.0.10.0/24", "10.0.11.0/24", "10.0.12.0/24"]
  availability_zones   = ["us-east-1a", "us-east-1b", "us-east-1c"]
  project              = var.project
  tags                 = var.tags
}

module "compute" {
  source = "./modules/compute"

  vpc_id     = module.networking.vpc_id          # Output from networking
  subnet_ids = module.networking.private_subnet_ids
}
```

Versioning (for shared modules):

```
# From Git repository
module "networking" {
  source = "git::https://github.com/myorg/terraform-modules.git//networking?
ref=v1.2.0"
}

# From Terraform Registry
module "vpc" {
  source  = "terraform-aws-modules/vpc/aws"
  version = "~> 5.0"
}
```

Module Best Practices:

- Every module has `variables.tf`, `outputs.tf`, `main.tf`
- Use `description` on every variable
- Use `type` constraints and `validation` blocks
- Pin versions with `~>` (pessimistic constraint)
- Test modules with `terraform plan` before publishing
- Use `for_each` over `count` for better state management

This concludes Part 2. Continue to Part 3 for Docker/Kubernetes, CI/CD, Python Testing, and Architecture Design sections.